

# VeriCase Consolidated Technical Architecture and Strategy Report

## 1. Introduction and Market Positioning

VeriCase is an AI-assisted case evidence management platform designed to streamline how legal teams, particularly those in the construction sector, ingest, organise, and analyse case-related communications and documents.

This report provides a comprehensive overview of VeriCase's confirmed design decisions, system architecture, technology stack, competitive positioning, MVP status, and the roadmap for full production release.

### 1.1 Market Opportunity and Competitive Landscape

VeriCase sits at the unique intersection of the \$5.8 billion eDiscovery market and the \$13.7 billion construction software market. Analysis of 20+ competitors reveals a significant gap:

**Legal eDiscovery platforms** (e.g., Relativity, Everlaw) struggle with cost at scale, often charging \$7,000-10,000 monthly for a typical 500GB construction dataset.

**Legal Practice Management tools** (e.g., Clio, MyCase) lack massive data ingestion capabilities and critically fail to offer native PST (email archive) file handling.

**Construction Platforms** (e.g., Procore, Aconex) excel at project management but lack specialized legal features for evidence preparation or dispute resolution.

Given that 91% of construction professionals experienced disputes in 2023, VeriCase's ability to bridge these sectors with cost-effective massive data handling capabilities presents a significant market opportunity.

### 1.2 Key Differentiators

VeriCase's competitive advantages crystallize around five key differentiators:

**Cost Structure for Massive Data:** Aiming for significant savings compared to competitors, a decisive factor for data-heavy construction disputes.

**Native PST File Processing:** Specialized handling for 10-50GB mail archives, including automatic thread extraction.

**Construction-Legal Integration:** Uniquely combines legal tech features (evidence bundling) with construction data understanding.

**AI-Powered Search with Context:** Semantic understanding specifically tuned for construction disputes—connecting events, communications, and outcomes for cause-effect analysis (Detailed in Section 5.3).

**Unified Collaboration Platform:** Enables construction teams and legal counsel to work from the same dataset securely.

## 2. End-to-End Architecture Overview

VeriCase's architecture follows a modular end-to-end pipeline built using modern, proven technologies.

### *Ingestion Layer*

Modules pull in raw evidence. The MVP focuses on reading raw email files (PST, MBOX, EML) and automatically parsing them into a structured format.

### *Storage Layer*

**PostgreSQL** serves as the source of truth for structured information (cases, messages, tags). A specialised **vector database (Pinecone for MVP)** stores high-dimensional embeddings for semantic search. Files (attachments) are stored in cloud object storage (e.g., AWS S3).

### *Application Logic Layer*

The core logic is split. A **Node.js** server (Express.js) serves as the main API layer. **Python** is used for specialised processing tasks benefiting from its rich AI and parsing libraries (e.g., parsing emails, generating embeddings).

### *AI Integration*

Integration with **OpenAI's API** (GPT-4 and embedding models) is central. Used for (1) generating vector embeddings of content and (2) vectorizing natural language queries to match against the index by meaning rather than keywords.

### *Interface Layer*

A **React** application (JavaScript/TypeScript) provides the user interface. It communicates with the Node.js backend via RESTful APIs.

### *Deployment Layer*

All components are containerised using **Docker**. The architecture is cloud-native, targeting container orchestration (like AWS ECS or Kubernetes) for scaling. CI/CD pipelines are integrated.

## 3. Technology Stack

**Backend Database:** PostgreSQL 15.

**Vector Search Database:** Pinecone (hosted service for MVP). Alternatives like Weaviate considered for future self-hosting.

**Backend Servers:** Node.js 18 (Express.js) for API; Python 3.10+ for processing/AI services.

**Front-End:** React 18 (with TypeScript). D3.js or Chart.js expected for visualizations.

**AI and NLP:** OpenAI API (GPT-4 and embedding models).

**Email/Message Parsing:** Python libraries (e.g., mail-parser), following eDiscovery best practices for threading.

**Deployment/DevOps:** Docker, Git, CI Pipelines (e.g., GitHub Actions). Target hosting on AWS (ECS, RDS, S3).

## 4. Completed MVP Functionality

The VeriCase MVP includes the following working features demonstrating end-to-end capabilities:

### **Case/Project Setup Wizard**

A guided React process for creating a new case, entering metadata, and uploading initial data sources.

### **Email Extraction and Message Thread Parsing**

Ingests raw email data and automatically organizes it into conversation threads. It reduces review time by focusing on the last inclusive message containing prior context, following eDiscovery standards.

### **AI-Driven Evidence Search (Backend)**

The backend semantic search engine is operational. The system creates vector embeddings and uses Pinecone to return results ranked by semantic relevance.

### **Tagging and Annotation**

Users can create and apply custom tags to any evidence item. The data model is prepared for future AI-assisted tagging.

### **Entity Extraction (Basic)**

Basic extraction of headers, participants, and entities from email signatures is implemented, laying the foundation for social graphs.

### **Data Visualisation Hooks**

The backend is prepared to compile data (e.g., interaction matrices, chronological lists) for timeline views and network graphs. Front-end implementation is pending.

## 5. Detailed Breakdown of Key Modules

This section details the technical implementation and practical function of major modules, integrating specific development plans.

### 5.1 Data Ingestion & Thread Parsing Module

**Technical Architecture:** Python-based module using libraries like *mail-parser*. Extracts data into PostgreSQL. Implements a threading algorithm using Message-IDs, In-Reply-To headers, and subject analysis. Flags 'inclusive' emails.

### 5.2 Case/Project Setup Module

**Technical Architecture:** Multi-step React form backed by Node.js APIs. Manages metadata input, validation, database creation, and triggers the ingestion module via a background job queue upon file upload.

### 5.3 AI-Driven Evidence Search Module (Integrated Design)

This module provides the intelligent search capability for the case. It spans backend semantic search logic and an advanced front-end Natural Language Understanding (NLU) interface.

#### *Backend Architecture (Semantic Engine - MVP Complete)*

The core combines OpenAI embeddings and the Pinecone vector database.

**Indexing:** Ingested text is embedded via OpenAI and upserted into Pinecone, linked to the PostgreSQL record.

**Querying:** User queries are embedded in real-time and matched against Pinecone using nearest neighbor search for semantic similarity.

**Retrieval:** The service fetches metadata for top results from PostgreSQL. (RAG for direct answers is planned post-MVP).

#### *Frontend Architecture and NLU (Master Data Query Design - Planned)*

The user interface is a single command bar above the master data table (Cmd+K opens). It features a lean, fast, and auditable dual-mode design built for million-item scale.

#### **Dual Modes:**

**1. Filter Mode:** Turns plain English into precise table filters (e.g., 'emails from the client in Q1 2022'). Updates the table view only.

**2. Evidence Mode:** Asks a factual question (e.g., 'find everything about the loading bay'). Uses semantic search and opens a right drawer with ranked results and explainers.

#### **NLU Parser and Query Plan (Crucial Feature):**

A specialized parser extracts facets from natural language. A compact "Query Plan" pill shows exactly what the system understood *before* execution. Clicking it reveals all parsed facets as editable chips, preventing silent errors and allowing instant correction.

#### *Parser Contract:*

Input: free text, party dictionary, document types, timezone.

Output: structured facets with confidence scores, plus unresolved tokens.  
*Pseudocode Sketch (Filter Mode Parser):*

```
def parse_to_facets(text, party_dict, tz):
    facets = {}
    entities = ner_extract(text) # parties, dates, channels, directions, keywords
    # ... [Facet extraction logic] ...
    a, b = resolve_parties(entities.parties, party_dict)
    facets["from"], facets["to"] = a, b
    facets["date_range"] = normalize_dates(entities.dates, tz)
    return facets, unresolved_tokens(entities)
```

#### **Performance and Safety:**

Evidence mode insists on context (slot filling). A broad query like "loading bay" will prompt the user to select a period or stakeholders before executing, keeping it fast and accurate. Every plan is logged for audit.

#### **Acceptance Criteria (Developer Guidance):**

Free text 'emails from client to United Living between Jan 2021 and Feb 2022' must produce four chips matching those facets and yield the same result set as manual filters.

Evidence query "loading bay" must not execute until at least one period or stakeholder is chosen.

Every executed plan must appear in the audit log with facets, user, timestamp, result count, and duration.

### **5.4 Tagging & Annotation Module**

**Technical Architecture:** Uses a 'Tags' table and a join table ('Evidence\_Tags') in PostgreSQL. Provides CRUD APIs for tag management. Architecture supports future AI auto-tagging.

### **5.5 Data Visualisation & Analytics Module**

**Technical Architecture:** Backend logic is ready to aggregate data for visual components (Timeline and Network Graphs). Endpoints exist to provide interaction matrices and chronological lists. Front-end implementation (D3.js/Chart.js) is pending.

## 6. Outstanding Modules and Features for Full Delivery

**Comprehensive Data Visualisation UI:** Building the interactive front-end components (Timeline, Network Graph).

**User Management & Access Control:** Implementing authentication (login/SSO) and role-based access control (RBAC).

**Advanced Document Ingestion:** Expanding beyond email to handle PDFs, Word files, images (with OCR), and chat transcripts.

**AI-Based Summaries & Insights:** Implementing auto-summarisation, advanced entity extraction/linking (NLP), and anomaly detection.

**Performance Scaling and Hardening:** Optimizations for scale (pagination, indexing, load testing, caching) and improving error handling/recovery.

**Security and Compliance Features:** Implementing data encryption (at rest/in transit), audit logging, and data retention policies.

**UI/UX Refinements and Collaboration Tools:** Commenting/notes, export functions (PDF/Excel), and general polish.

## 7. Task Complexity Breakdown

### Tasks Requiring Senior Development Skills

Architecture & Scaling Design; Security & Compliance Implementation (SSO, Encryption); Complex AI Feature Integration (RAG flows, model tuning); Advanced Data Visualisation (D3.js performance); DevOps Strategy (Infrastructure-as-code, Orchestration).

### Standard Development Tasks (Mid-level)

Front-End Feature Development (React components, forms); CRUD API Endpoints (Node.js); Simple Database Schema Design; Unit/Integration Testing and Bug Fixing; Basic API Integrations (e.g., OCR services); UI/UX Polishing.

### Suitable for AI-Assisted Coding (e.g., Copilot)

Boilerplate Code Generation (DTOs, ORM definitions); Unit Test Scaffolding; Data Conversion Scripts; Simple Utility Functions.

## 8. Development Timeline and Sprint Estimates

Remaining development work is estimated based on two-week sprints. The realistic timeline from the current MVP to a fully delivered product is approximately 3 to 4 months (6-8 sprints) of active development by a small team.

### 8.1 Immediate Focus: AI Master Data Query UI (3-Week Plan)

The immediate priority is implementing the dual-mode AI Search interface detailed in Section 5.3.

#### *Week One*

React command bar (modes, chips, Query Plan pill, keyboard bindings).

Natural language to facets parser (dates, parties, direction, type).

Server-side filter execution and persistence.

#### *Week Two*

Evidence mode slot filling (chip questions).

Hybrid retrieval service integration (lexical plus vector search) and simple reranker.

Right drawer results with explainers and adjustable chips.

#### *Week Three*

Telemetry and audit events.

Saved searches and shareable links.

Edge cases and language variants (e.g., "last winter", "past three months").

### 8.2 Overall Roadmap Estimates

| Feature/Module                          | Status         | Estimated Sprints (2 weeks) |
|-----------------------------------------|----------------|-----------------------------|
| Case Setup & Ingestion (Email)          | Complete (MVP) | 0                           |
| AI Semantic Search (Backend)            | Complete (MVP) | 0                           |
| Tagging (Manual)                        | Complete (MVP) | 0                           |
| AI Search UI (NLU/Frontend)             | Planned        | 1.5 (See 8.1)               |
| Data Visualisation (Timeline & Network) | Planned        | 2                           |
| User Auth & Access Control              | Planned        | 1-2                         |
| Advanced Ingestion (PDF, Chat)          | Planned        | 2                           |
| AI Summarisation & NLP Enhancements     | Planned        | 1                           |
| Performance Tuning and Scalability      | Planned        | 1 (ongoing)                 |
| Security Hardening & Polish             | Planned        | 2                           |